

Lecture 4: Dynamic Programming



Happy Support Latino Business Day

- Other Holidays:
 - National Coloring Day
 - National Sober Day
 - The Exaltation of the Holy Cross
 - National Cream-Filled Doughnut Day
 - National Boss/Employee Exchange Day
 - National Black and White Cookie Day
 - Eat a Hoagie Day
 - I'm on Top of It Day
 - National Parents Day Off
 - Gobstopper Day
 - National Live Creative Day
- This week is National Vegetarian Awareness Week
- September is Classical Music Month

Dynamic Programming

High-level Idea: Break a problem into smaller subproblems (like divide-and-conquer) but divide into **many overlapping** subproblems.

The DP technique applies to problems that obey the **principle of “optimal substructure”**: the overall optimal solution can be constructed from optimal solutions to smaller subproblems

Warm-Up: Fibonacci

Input: non-negative integer n

Output: n^{th} Fibonacci number

Recurrence for Fibonacci: $F(n) = F(n-1) + F(n-2)$
 $F(0) = F(1) = 1$

Given a recurrence, we will explore three ways to compute its values

Fibonacci Method 1: Top-down Recursive

$$F(n) = F(n-1) + F(n-2)$$
$$F(0) = F(1) = 1$$

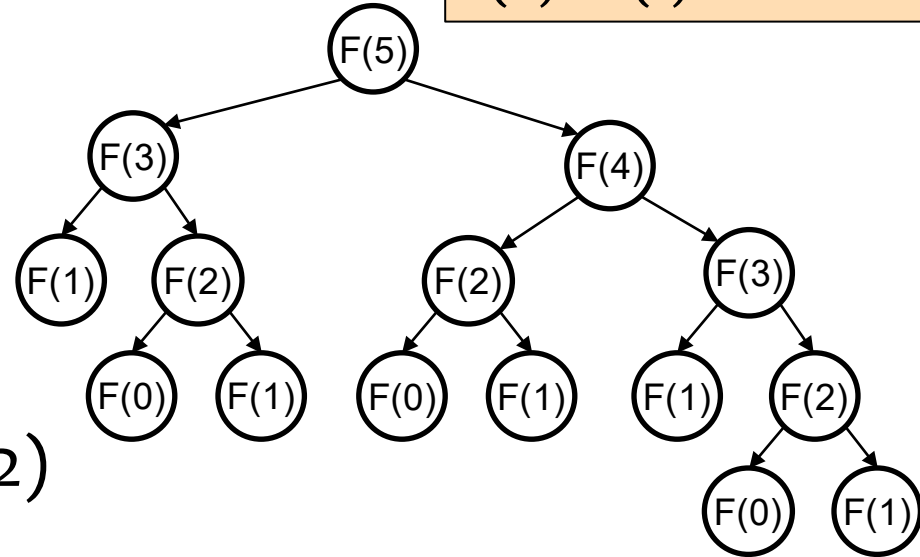
Algorithm Fib(n):

If $n = 0$ OR $n = 1$:

Return 1

Else:

Return $\text{Fib}(n-1) + \text{Fib}(n-2)$



- **Pro:** direct translation of recurrence into code
- **Con:** $\exp(n)$ recursive calls

Fibonacci Method 2: Top-down with Memoization

$$F(n) = F(n-1) + F(n-2)$$
$$F(0) = F(1) = 1$$

memo := array indexed from 0 to n

Algorithm Fib(n):

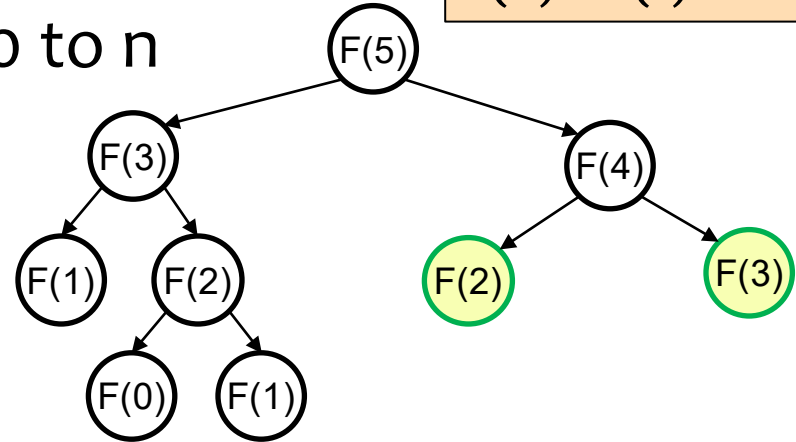
If $n = 0$ OR $n = 1$:

Return 1

Else if **memo**(n) is empty:

memo(n) = Fib(n-1) + Fib(n-2)

Return memo(n)



- **Pro:** way faster than method 1
- **Con:** not clear how to analyze running time

Fibonacci Method 3: Bottom-Up Table Filling

Algorithm Fib(n):

table := array indexed from 0 to n

table(0) = 1

table(1) = 1

for i = 2 to n:

table(i) = **table**(i-1) + **table**(i-2)

Return table(n)

This still isn't poly time since the input size is $\log n$, but this **method** gives poly time algorithms for many other problems



- **Pros:** gives roadmap for running time analysis:
(size of table) $\cdot$ (time to fill each entry)
- **Con:** need to translate from recurrence to filling table bottom-up

Warm-Up: Fibonacci

Input: non-negative integer n

Output: n^{th} Fibonacci number

Recurrence for Fibonacci: $F(n) = F(n-1) + F(n-2)$
 $F(0) = F(1) = 1$

Given a recurrence, three ways to compute its values:

1. **Top-down Recursive:** Starting at desired input, recurse down to base case(s)
2. **Top-down with Memoization:** Same as 1, but save results as they're computed, reusing already-computed results
3. **Bottom-Up Table Filling:** Start from base case(s), build up to desired result

Now let's solve a problem using Dynamic Programming!

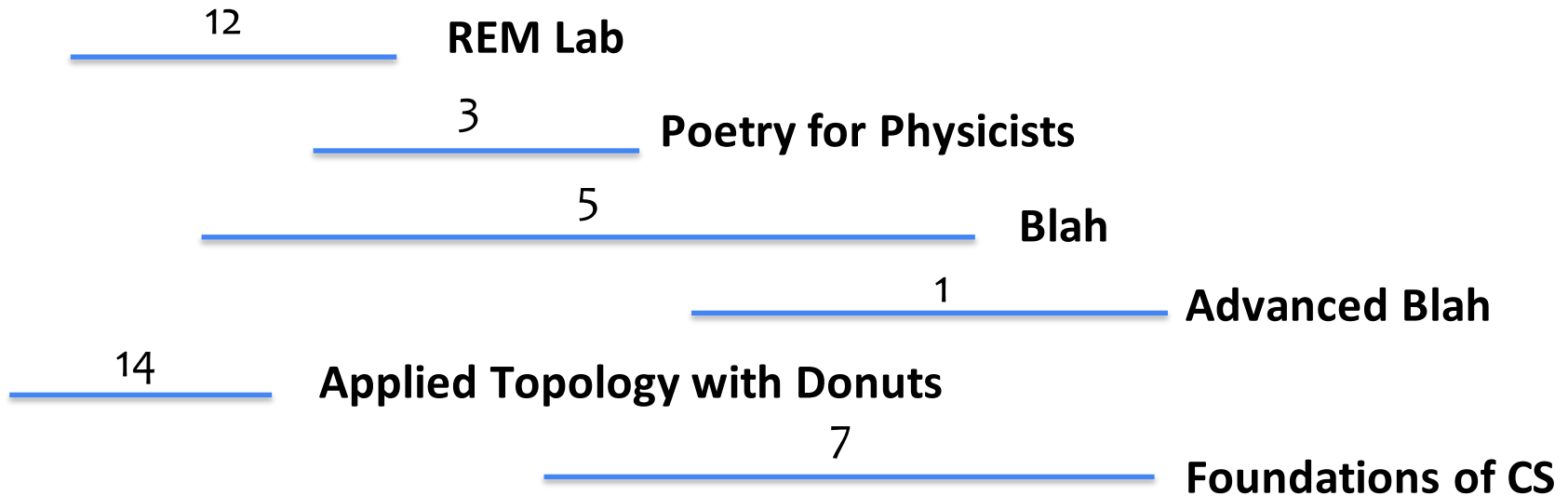
What we'll do:

1. Derive a recurrence to express the solution value
2. Compute the values of the recurrence using method 3 (bottom-up table filling).
3. Reconstruct a solution from the recurrence values

*Method 2 (top-down with memoization) is also considered “dynamic programming” but we'll **only** focus on method 3 (bottom-up table filling) since it helps us calculate the running time.

Weighted Course Registration Problem

(aka Weighted Task Selection)



Goal: Choose a set of non-intersecting courses with largest total value.
(there may be many optimal solutions, we just seek one)

I'd take the donuts course if it didn't create a hole in my schedule!



*Let the input size be $n = \text{\#intervals}$. (Assume the weights are small enough that we can disregard their contribution to the input size.)

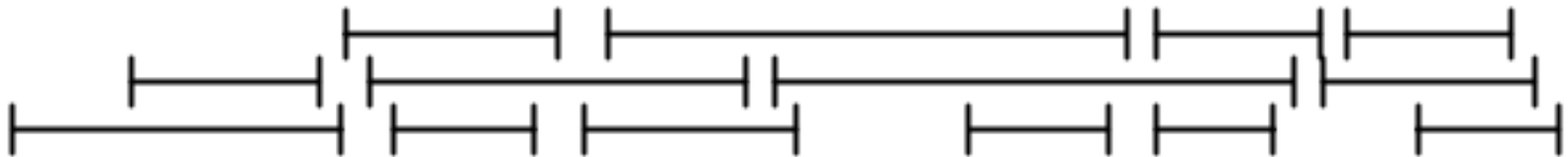
Weighted Task Selection Recurrence

Assume the intervals $J_1, J_2, \dots, J_n$ are sorted by _____ time.

Let OPT be an optimal solution. Start from J_n and reason backwards.

There are two options:

- OPT has J_n (use it!)
- OPT doesn't have J_n (lose it!)



Weighted Task Selection Recurrence

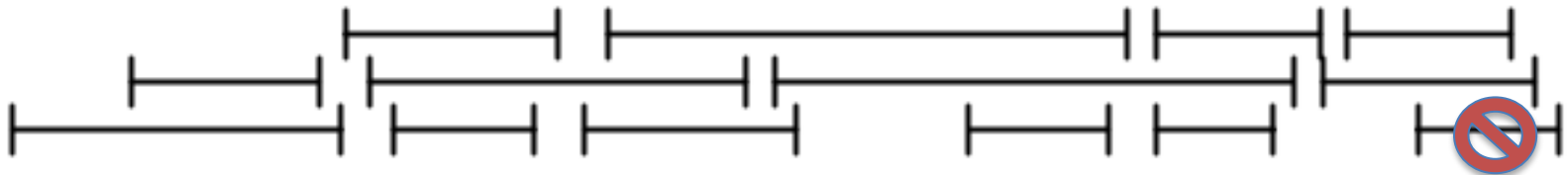
Assume the intervals $J_1, J_2, \dots, J_n$ are sorted by _____ time.

Let OPT be an optimal solution. Start from J_n and reason backwards.

There are two options:

- OPT has J_n (use it!)
- OPT doesn't have J_n (lose it!) $\leftarrow \text{OTV}(J_1, \dots, J_n) = \text{OTV}(J_1, \dots, J_{n-1})$

“Optimal Task Value” i.e. the **value** of the optimal solution



Weighted Task Selection Recurrence

Assume the intervals $J_1, J_2, \dots, J_n$ are sorted by finish time.

Let OPT be an optimal solution. Start from J_n and reason backwards.

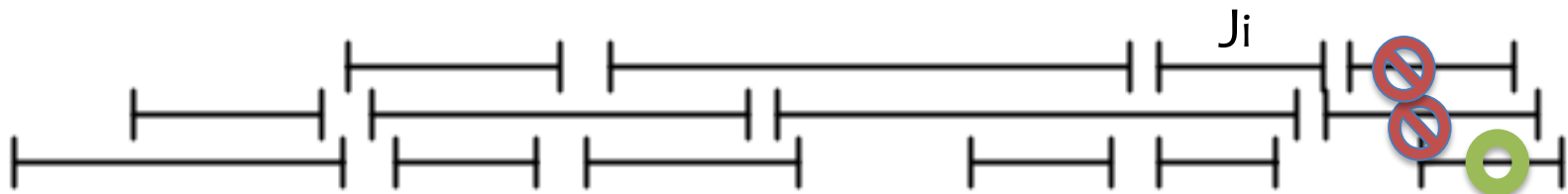
There are two options:

- OPT has J_n (**use it!**) $\leftarrow \text{OTV}(J_1, \dots, J_n) = \text{val}(J_n) + \text{OTV}(J_1, \dots, J_i)$

J_i is the last interval that doesn't overlap with J_n

- OPT doesn't have J_n (**lose it!**) $\leftarrow \text{OTV}(J_1, \dots, J_n) = \text{OTV}(J_1, \dots, J_{n-1})$

"Optimal Task Value" i.e. the **value** of the optimal solution



The Final Recurrence

$$\text{OTV}(J_1, \dots, J_n) = \max \left\{ \begin{array}{ll} \text{val}(J_n) + \text{OTV}(J_1, \dots, J_i), & \text{(use it!)} \\ \text{OTV}(J_1, \dots, J_{n-1}) & \text{(lose it!)} \end{array} \right\}$$

i = index of last interval (before J_n) that doesn't overlap with J_n

You could write code to implement this recurrence as an algorithm...

Algorithm OTV($J_1, \dots, J_n$):

if $n = 0$:

Return 0

else:

i = index of last interval (before J_n) that doesn't overlap with J_n

Return $\max\{\text{val}(J_n) + \text{OTV}(J_1, \dots, J_i), \text{OTV}(J_1, \dots, J_{n-1})\}$

... but it would run in exponential time

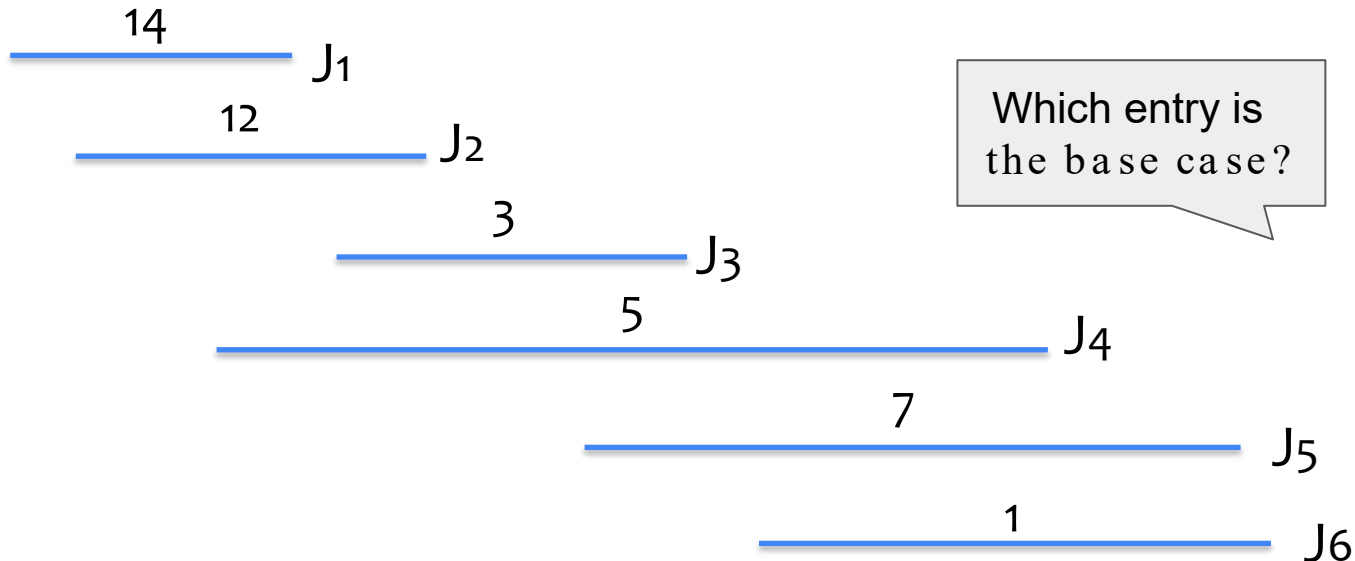
Dynamic Programming in Action!

(use it)

(lose it)

For reference: $\text{OTV}(J_1, \dots, J_n) = \max\{\text{val}(J_n) + \text{OTV}(J_1, \dots, J_i), \text{OTV}(J_1, \dots, J_{n-1})\}$

$\text{OTV}(\emptyset)$	$\text{OTV}(J_1)$	$\text{OTV}(J_1, J_2)$	$\text{OTV}(J_1, \dots, J_3)$	$\text{OTV}(J_1, \dots, J_4)$	$\text{OTV}(J_1, \dots, J_5)$	$\text{OTV}(J_1, \dots, J_6)$



Which entry is the base case?

Which cell contains the final answer?



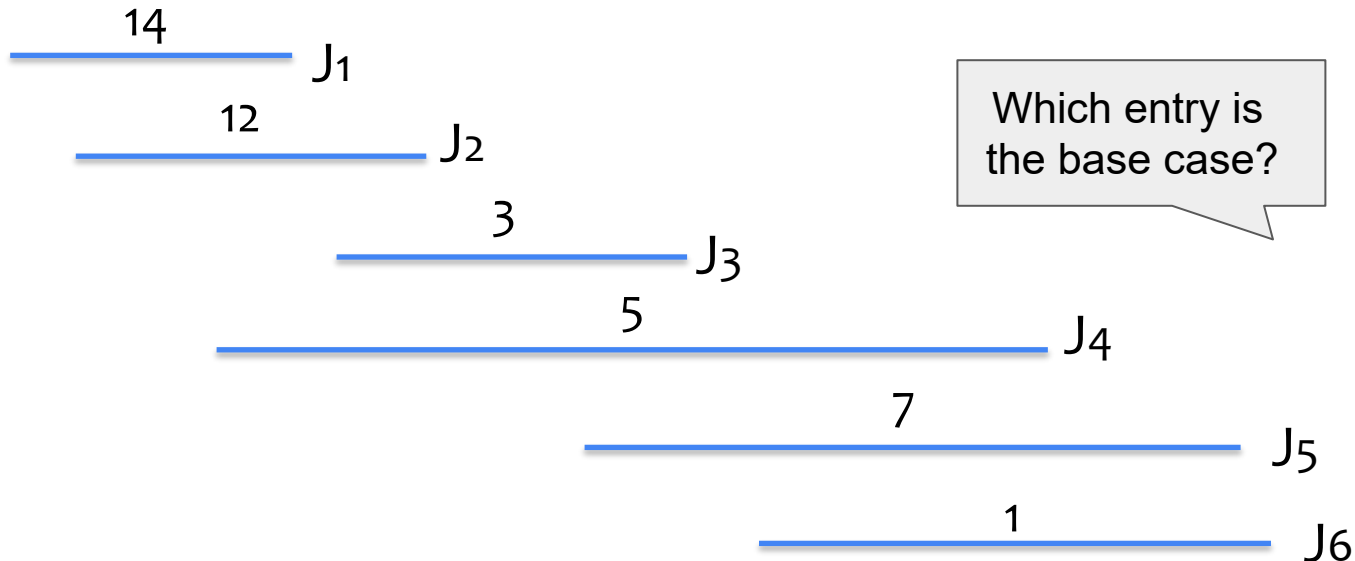
Dynamic Programming in Action!

(use it)

(lose it)

For reference: $\text{OTV}(J_1, \dots, J_n) = \max\{\text{val}(J_n) + \text{OTV}(J_1, \dots, J_i), \text{OTV}(J_1, \dots, J_{n-1})\}$

$\text{OTV}(\emptyset)$	$\text{OTV}(J_1)$	$\text{OTV}(J_1, J_2)$	$\text{OTV}(J_1, \dots, J_3)$	$\text{OTV}(J_1, \dots, J_4)$	$\text{OTV}(J_1, \dots, J_5)$	$\text{OTV}(J_1, \dots, J_6)$
0						



Which entry is the base case?

Which cell contains the final answer?



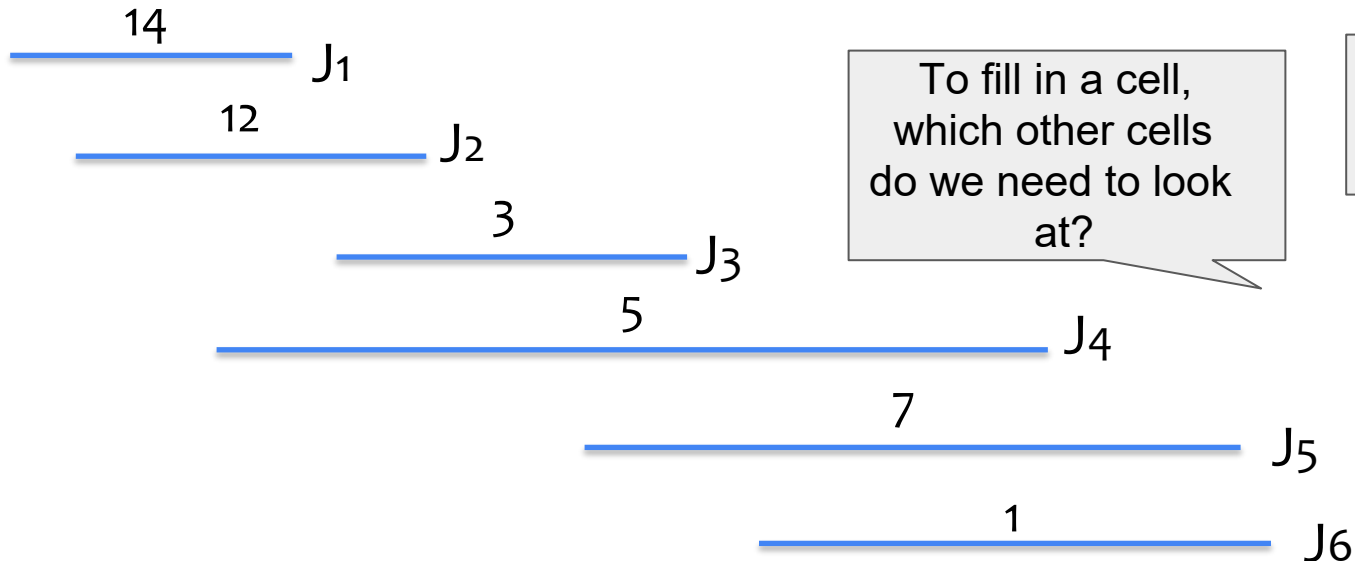
Dynamic Programming in Action!

(use it)

(lose it)

For reference: $\text{OTV}(J_1, \dots, J_n) = \max\{\text{val}(J_n) + \text{OTV}(J_1, \dots, J_i), \text{OTV}(J_1, \dots, J_{n-1})\}$

$\text{OTV}(\emptyset)$	$\text{OTV}(J_1)$	$\text{OTV}(J_1, J_2)$	$\text{OTV}(J_1, \dots, J_3)$	$\text{OTV}(J_1, \dots, J_4)$	$\text{OTV}(J_1, \dots, J_5)$	$\text{OTV}(J_1, \dots, J_6)$
0						



To fill in a cell,
which other cells
do we need to look
at?

In which order do
we fill the table?



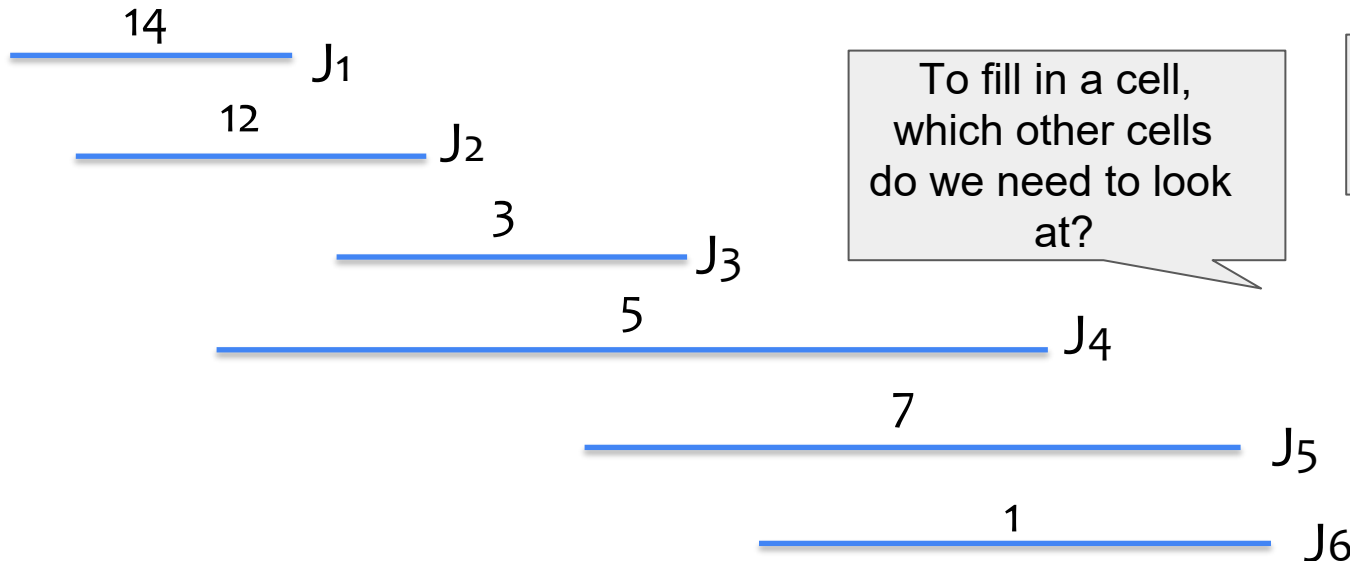
Dynamic Programming in Action!

(use it)

(lose it)

For reference: $\text{OTV}(J_1, \dots, J_n) = \max\{\text{val}(J_n) + \text{OTV}(J_1, \dots, J_i), \text{OTV}(J_1, \dots, J_{n-1})\}$

$\text{OTV}(\emptyset)$	$\text{OTV}(J_1)$	$\text{OTV}(J_1, J_2)$	$\text{OTV}(J_1, \dots, J_3)$	$\text{OTV}(J_1, \dots, J_4)$	$\text{OTV}(J_1, \dots, J_5)$	$\text{OTV}(J_1, \dots, J_6)$
0	14	14	17	17	21	21



To fill in a cell,
which other cells
do we need to look
at?

In which order do
we fill the table?



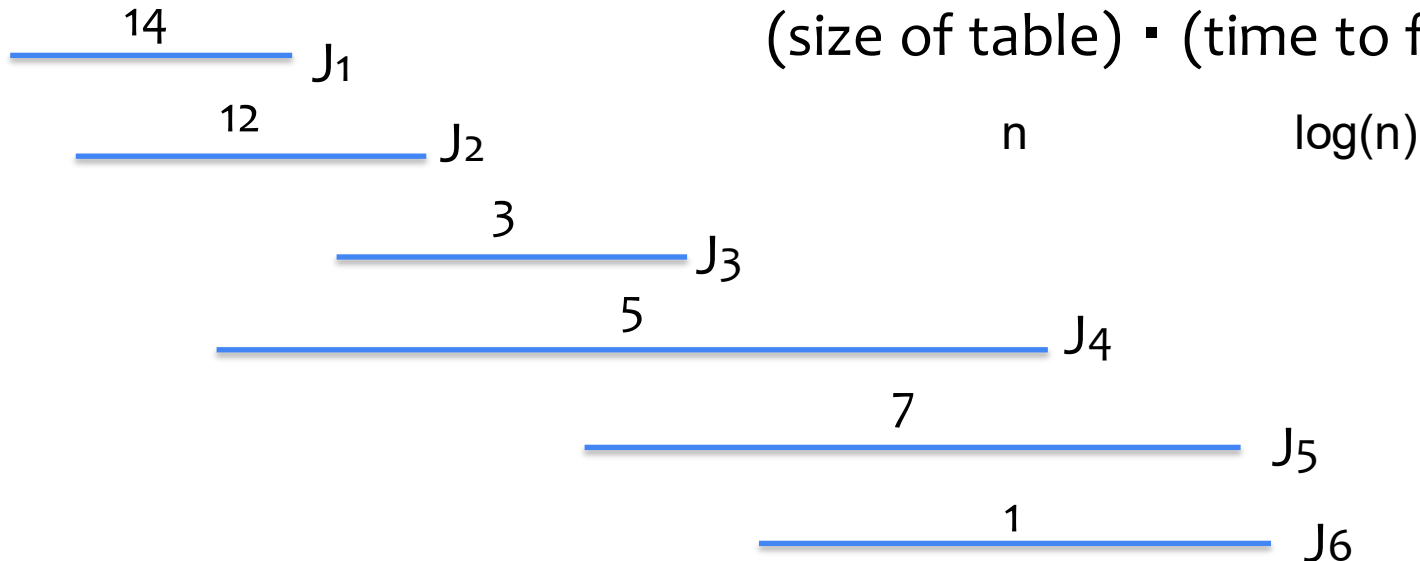
Dynamic Programming in Action!

For reference: $\text{OTV}(J_1, \dots, J_n) = \max^{(\text{use it})}_{\text{(lose it)}} \{ \text{val}(J_n) + \text{OTV}(J_1, \dots, J_i), \text{OTV}(J_1, \dots, J_{n-1}) \}$

OTV($\emptyset$)	OTV(J_1)	OTV(J_1, J_2)	OTV($J_1, \dots, J_3$)	OTV($J_1, \dots, J_4$)	OTV($J_1, \dots, J_5$)	OTV($J_1, \dots, J_6$)
0	14	14	17	17	21	21

Running time =

(size of table) $\cdot$ (time to fill each entry)



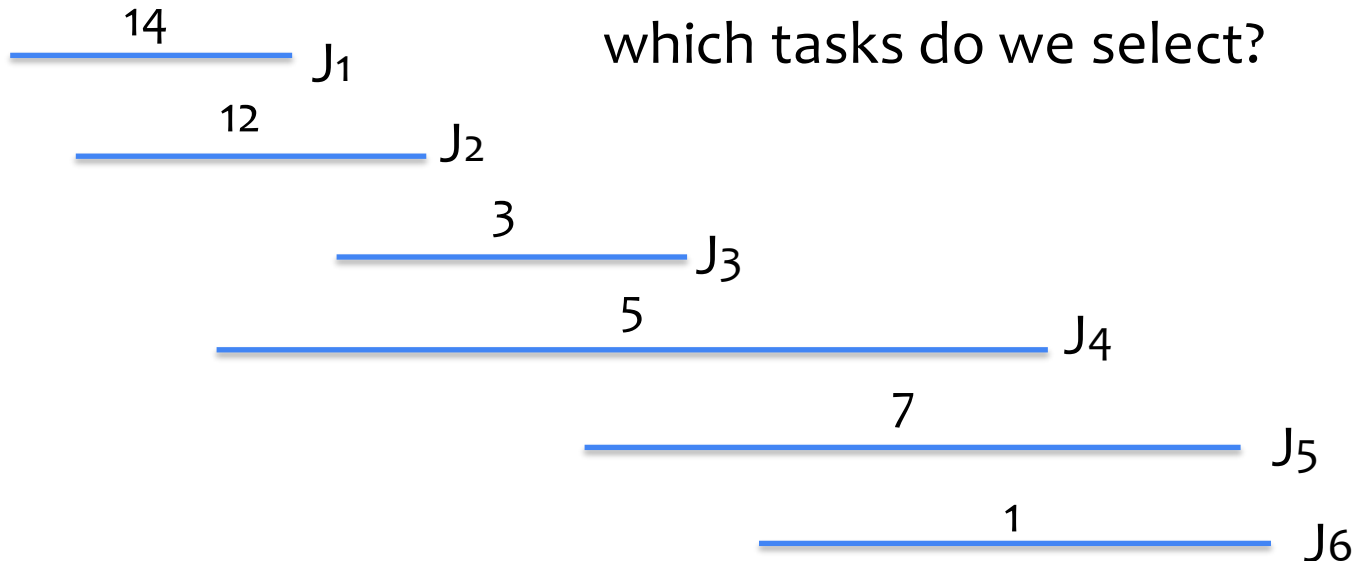
Dynamic Programming in Action!

For reference: $\text{OTV}(J_1, \dots, J_n) = \max\{\text{val}(J_n) + \text{OTV}(J_1, \dots, J_i), \text{OTV}(J_1, \dots, J_{n-1})\}$

(use it) (lose it)

$\text{OTV}(\emptyset)$	$\text{OTV}(J_1)$	$\text{OTV}(J_1, J_2)$	$\text{OTV}(J_1, \dots, J_3)$	$\text{OTV}(J_1, \dots, J_4)$	$\text{OTV}(J_1, \dots, J_5)$	$\text{OTV}(J_1, \dots, J_6)$
0	14	14	17	17	21	21

How to reconstruct the solution, i.e.,
which tasks do we select?

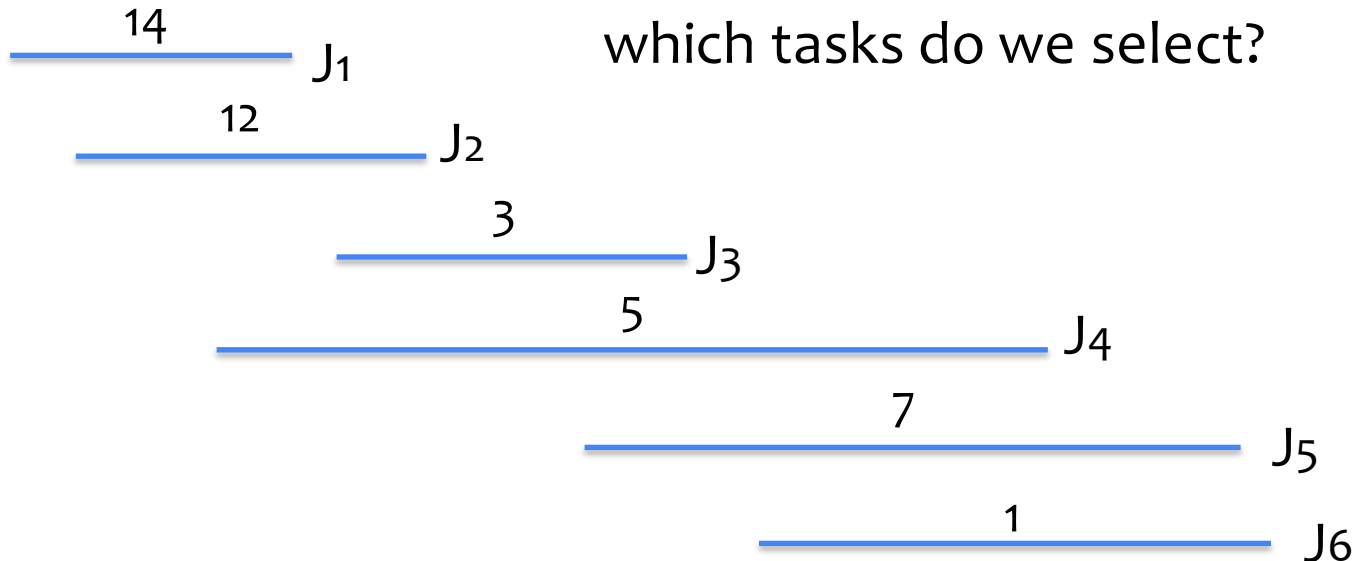


Dynamic Programming in Action!

For reference: $\text{OTV}(J_1, \dots, J_n) = \max\{\overset{\text{(use it)}}{\text{val}(J_n) + \text{OTV}(J_1, \dots, J_i)}, \overset{\text{(lose it)}}{\text{OTV}(J_1, \dots, J_{n-1})}\}$

$\text{OTV}(\emptyset)$	$\text{OTV}(J_1)$	$\text{OTV}(J_1, J_2)$	$\text{OTV}(J_1, \dots, J_3)$	$\text{OTV}(J_1, \dots, J_4)$	$\text{OTV}(J_1, \dots, J_5)$	$\text{OTV}(J_1, \dots, J_6)$
0	14	14	17	17	21	21

How to reconstruct the solution, i.e.,
which tasks do we select?



usually the trickiest part

The DP Recipe

1. Define *value* version of what we want to compute
2. Derive a recurrence *“Let $OTV(J_1, \dots, J_n)$ be...”*
3. Size of table: How many dimensions? Range of each dimension?
4. What are the base case(s)?
5. To fill in a cell, which other cells need to be filled already? In which order do I fill the table?
6. Which cell(s) contain the final answer?
7. Running time is often (size of table) $\cdot$ (time to fill each entry)
8. To reconstruct the solution (instead of just its value) follow arrows from final answer to base case

The Pseudocode

// returns the value of the optimal tasks  step 1 of DP recipe

Algorithm OTV($J_1, \dots, J_n$):

table := array indexed from 0 to n  step 3 of DP recipe

table(0) = 0  step 4 of DP recipe

for k = 1 to n:  step 5 of DP recipe

 i = index of last interval (before J_k) that doesn't overlap with J_k

table(k) = $\max\{\text{val}(J_k) + \text{table}(i), \text{table}(k-1)\}$

Return table(n)  step 6 of DP recipe  steps 2,5 of DP recipe

Structure of **all** DP pseudocode: loop through the table in appropriate order and fill in cells according to the recurrence.

⇒ After answering DP Recipe questions, no creativity is needed!

Another example of DP:

Longest Increasing Subsequence (LIS)

Input: Array A of n numbers

Output: A longest increasing subsequence of A

Example: What is an LIS of [1,4,3,7,2]?

Recurrence for LIS

It's easy! Check it out! $\text{LIS}(A[1]) = 1$

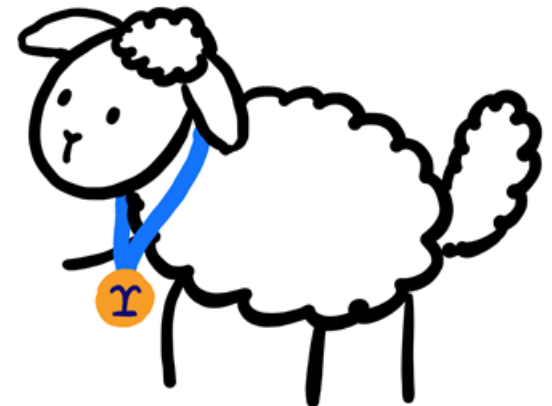
$$\text{LIS}(A[1..n]) = \begin{cases} \text{LIS}(A[1..n-1]) + 1 & \text{if } A[n] > A[n-1] \text{ (use it!)} \\ \text{LIS}(A[1..n-1]) & \text{otherwise (lose it!)} \end{cases}$$

Does this work?

No! Counterexample (one of MANY):

[8, 9, 4, 5, 6, 10, 7, 1, 2, 3, 9]

LIS so far: 1 2 2 2 3 4 4 4 4 4 5



Recurrence for LIS

Defn: Let $\text{END-LIS}(A[1..i])$ be the length of an LIS of $A[1..i]$ *that ends in $A[i]$* .

What is $\text{END-LIS}(A[1..i])$ for each i in the previous example?

$A = [8, 9, 4, 5, 6, 10, 7, 1, 2, 3, 9],$

End-LIS: 1 2 1 2 3 4 4 1 2 3 5

Recurrence:

$\text{END-LIS}(A[1..i]) = 1 + \max\{\text{END-LIS}(A[1..j]) \text{ among all } j < i \text{ with } A[j] < A[i]\}$
(if such j exists)

Base case: $\text{END-LIS}(A[1..i]) = 1$ if $A[i]$ is the smallest element so far in A

Let's follow the DP Recipe

$\text{END-LIS}(A[1..i]) = 1 + \max\{\text{END-LIS}(A[1..j]) \text{ among all } j < i \text{ with } A[j] < A[i]\}$

A =	3	7	1	4	6	9	2	5
END-LIS(A[1..i])								

1. Define value version of what we want to compute
2. Derive a recurrence
3. Size of table: How many dimensions? Range of each dimension?
4. What are the base case(s)?
5. To fill in a cell, which other cells need to be filled already? In which order do I fill the table?
6. Which cell(s) contain the final answer?
7. Running time = (size of table) $\cdot$ (time to fill each entry)
8. To reconstruct the solution (instead of just its value) follow arrows

Let's follow the DP Recipe

$$\text{END-LIS}(A[1..i]) = 1 + \max\{\text{END-LIS}(A[1..j]) \text{ among all } j < i \text{ with } A[j] < A[i]\}$$

A =	3	7	1	4	6	9	2	5
END-LIS(A[1..i])	1	2	1	2	3	4	2	3

1. Define value version of what we want to compute
2. Derive a recurrence
3. Size of table: How many dimensions? Range of each dimension?
4. What are the base case(s)?
5. To fill in a cell, which other cells need to be filled already? In which order do I fill the table?

Could be any of them??
Scan for largest!
6. Which cell(s) contain the final answer?

n • n
7. Running time = (size of table) • (time to fill each entry)
8. To reconstruct the solution (instead of just its value) follow arrows

Pseudocode

Algorithm LIS($A[1..n]$) :

table := array indexed from 1 to n

for $i = 1$ to n :

if $A[i]$ is the minimum so far: // determined by scanning $A[1..i-1]$

table(i) = 1 // base case

Else:

table(i) = $1 + \max\{\mathbf{table}(j) \text{ among all } j < i \text{ with } \mathbf{A}[j] < \mathbf{A}[i]\}$

// find j by scanning **table** and **A**

Return $\max_i \mathbf{table}(i)$

What is the running time?